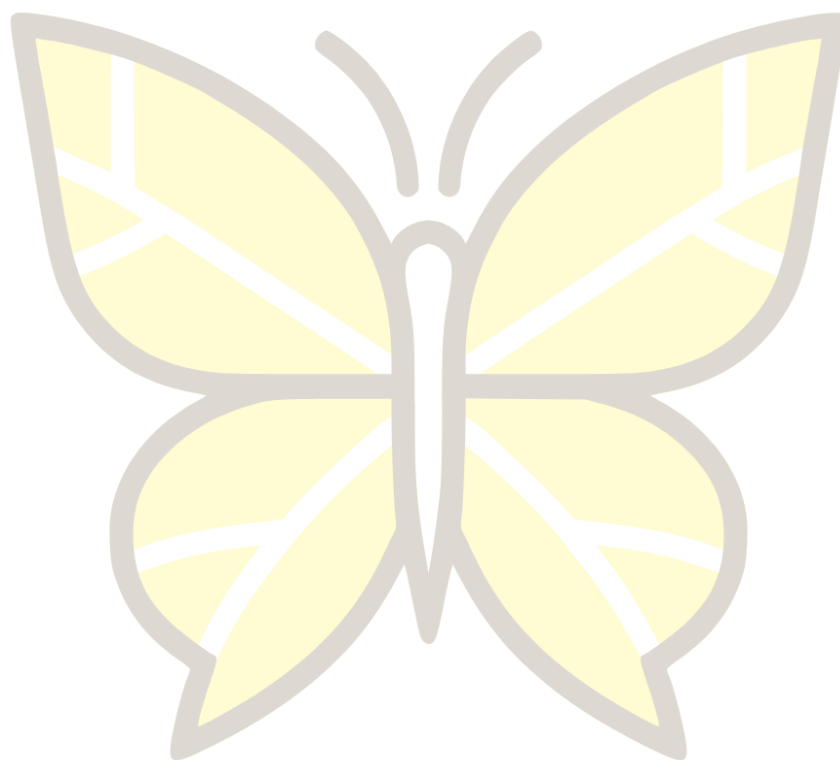


Intérprete de Claves Taxonómicas

Manuel Martín Santamaría



I.E.S. Comercio

Supervisor del TFG: María Cristina Azpiroz Merino

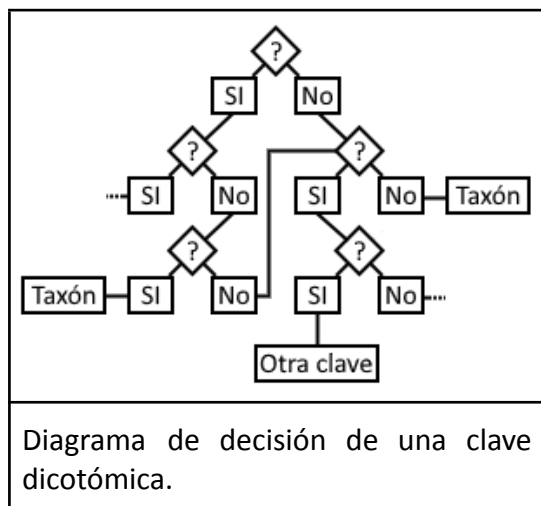
Índice

1 - Introducción.....	5
1.1 - ¿Qué es una clave taxonómica?.....	5
1.2 - Desventajas de las claves taxonómicas tradicionales.....	5
1.3 - Ventajas de mi propuesta.....	6
2 - Objetivos.....	7
2.1 Objetivo general.....	7
2.2 Objetivos específicos.....	8
3 - Fases del proyecto.....	8
3.1 - Planificación - (Análisis).....	10
3.1.1 - Análisis de los requisitos del software.....	10
3.1.1.1 - Requisitos funcionales.....	10
3.1.1.2 - Requisitos no funcionales.....	12
3.1.2 - Metodología de Desarrollo.....	13
3.1.2.1 - Descripción del marco de trabajo.....	13
3.1.2.2 - Adaptaciones Específicas.....	13
3.1.2.3 - Plataforma para la gestión de SCRUM.....	14
3.1.3- Análisis de las alternativas.....	14
3.1.3.1 - Tecnologías para el frontend (interfaz de usuario).....	14
3.1.3.2 - Tecnologías para el acceso a los datos.....	16
3.1.3.3 - Tecnologías de la interfaz REST.....	16
3.1.3.4 - Tecnologías para la base de datos.....	17
3.1.4 - Análisis de costos y beneficios.....	19
3.1.4.1 - Costos.....	19
3.1.4.2 - Beneficios.....	19
3.2 - Diseño.....	19
3.2.1 - Interfaz de usuario.....	19
3.2.1.1 - Descripción de la interfaz (capa de presentación).....	20
3.2.2 - API REST.....	20
3.2.2.1 - Descripción de la API REST (capa de lógica de Negocio).....	20
3.2.2.2 - Diagrama de clases.....	21
3.2.3 - Bases de datos.....	21
3.2.3.1 Bases de datos relacionales.....	21
3.2.3.2 Bases de datos no relacionales.....	22
3.2.5 - Arquitectura de hardware.....	23
3.2.6 - Arquitectura de software.....	25
3.2.7 - Patrones de diseño.....	26

3.2.8 - Esquema funcional.....	27
3.2.8.1 Componentes Principales.....	27
3.2.8.2 Diagrama del Esquema Funcional.....	27
3.2.8.3 Ejemplos del flujo de trabajo.....	28
3.3 - Implementación y configuración.....	29
3.3.1 - Gestión de las cargas de trabajo a través del proyecto.....	29
3.3.1.1 - Primer sprint (16/04/2024 - /30/04/2024) => 70 horas.....	29
3.3.1.2 - Segundo sprint (1/05/2024 - 14/05/2024) => 60 horas.....	29
3.3.1.3 - Tercer sprint (15/05/2024 - 29/05/2024) => 80 horas.....	30
3.3.1.4 - Otras tareas.....	30
3.3.1.5 - Vista general del proyecto.....	30
3.3.2 - Interfaz gráfica.....	31
3.3.3 - API REST.....	31
3.3.4 - Bases de datos.....	33
3.4 - Pruebas.....	33
4 - Posibles mejoras y ampliaciones.....	34
5 - Conclusión.....	34
6 - Bibliografía.....	35
7 - Anexos.....	37
7.1 - Diseño de la interfaz gráfica.....	37
7.2 - Paleta de colores.....	41
7.3 - Logotipo.....	41
7.4 - Normalización de las tablas de MariaDB.....	42
7.5 - Entornos y herramientas empleadas durante el proyecto.....	43
7.6 - Estructura de las clases de MongoDB y optimización de consultas.....	43
7.7 - Cómo ejecutar el proyecto.....	44
7.8 - Glosario.....	44

1 - Introducción

1.1 - ¿Qué es una clave taxonómica?



Las claves taxonómicas son documentos extensos empleados en biología para identificar especies o para determinar a qué grupo taxonómico pertenece un organismo.

Estas claves son conformadas por una sucesión de preguntas dicotómicas^[7], cuya respuesta suele ser en forma de afirmación o negación.

Cada respuesta desemboca en otra pregunta de forma sucesiva hasta culminar con la identificación del organismo o hasta referenciar a un taxón^[14] para el cual puede llegar a ser necesario consultar una clave diferente.

Estas claves taxonómicas se pueden basar en características visuales observables o anatomía, moleculares o metabólicas, basadas en características genéticas...

Para la aclaración de este concepto, un ejemplo de esta dinámica aparece representado en la figura.

1.2 - Desventajas de las claves taxonómicas tradicionales

La dificultad a la hora de interpretar estos volúmenes recae en el hecho de que contienen un vocabulario muy especializado y característico de cada grupo taxonómico (barreras lingüísticas). Además, hacen constantes menciones a términos o figuras cuya descripción o diagrama se encuentra en páginas distantes que hay que andar localizando de forma continua (constantes pérdidas de tiempo localizando la información).

En adición, los libros de claves taxonómicas tienden a quedarse desactualizados como consecuencia del descubrimiento de nuevas especies, o de la reorganización de los grupos taxonómicos como consecuencia de los resultados de investigaciones que emplean técnicas de biología molecular.

Además, las claves taxonómicas carecen de un formato interactivo, al ser mayormente textuales, lo que puede desalentar a los usuarios que se están introduciendo por primera vez en este mundo, lo que tiene un impacto negativo en el relevo generacional de esta especialización. Estos libros también suelen ser recursos caros o de difícil acceso, especialmente para investigadores con pocos recursos o para aficionados.

1.3 - Ventajas de mi propuesta

Las ventajas del intérprete de claves taxonómicas que planteo en este proyecto frente a las claves taxonómicas convencionales son:

- Mayor interactividad y facilidad de uso, brinda a los usuarios retroalimentación instantánea y orientación durante el proceso de identificación, facilitando el aprendizaje y reduciendo la posibilidad de error.
- Cualquier persona puede acceder a la herramienta a través de internet, eliminando las barreras de acceso al conocimiento científico.
- Las claves pueden ser modificadas, incluyendo nueva información en cualquier momento tras su creación, brindando datos más precisos y actualizados.
- Posibilidad de crear claves personalizadas para necesidades específicas, característica ideal para investigadores que trabajan con grupos taxonómicos menos comunes o en áreas particulares de estudio.
- El hecho de minimizar la necesidad de buscar a través de las diferentes páginas constantemente de manera constante, reduce el tiempo de consulta así como el riesgo de errores de identificación, lo que resulta en un proceso más eficiente y fiable.

2 - Objetivos

2.1 Objetivo general

La taxonomía es indispensable para el estudio y la conservación de la biodiversidad, es imperativo disponer de sistemas tecnológicos que, no solo faciliten el almacenamiento y procesamiento de grandes cantidades de datos, sino que a la vez faciliten su interpretación y acceso por parte de una amplia gama de usuarios.

El objetivo de este proyecto ha sido desarrollar una herramienta interactiva que, a través de una interfaz amigable elaborada con tecnologías web, agilice el proceso de consulta e interpretación de claves taxonómicas, mejorando a la vez la accesibilidad a las mismas por parte de usuarios sin conocimientos especializados.

La herramienta que planteo, no se centra exclusivamente en la interpretación, sino que también servirá para la generación de claves personalizadas, que esta misma herramienta pueda interpretar.

Considero que esta es una idea original, ya que no he encontrado herramientas de características similares. He encontrado alguna clave taxonómica interactiva (power points interactivos...), hechos de forma sencilla y específicos de alguna familia en concreto. No existe ninguna plataforma que sirva para interpretar claves taxonómicas, sin importar el taxón^[14], a la carta y de forma interactiva.

Esta solución fue presentada a doctores especializados en el campo de la entomología aplicada, estos la consideraron una propuesta muy interesante. Me transmitieron que sería de gran utilidad, tanto para gente que se está introduciendo en el mundo de la taxonomía, como para doctores especializados en una familia en particular y que se ven en la necesidad de aprender a identificar otros grupos taxonómicos.

Dicho esto, por medio de este proyecto también he tratado de profundizar en los conocimientos adquiridos durante este ciclo, así como ampliar mi conocimiento en áreas que no han sido tratadas en los módulos que lo componen, como en aspectos de seguridad, arquitectura de software, comunicaciones entre sistemas y optimización.

2.2 Objetivos específicos

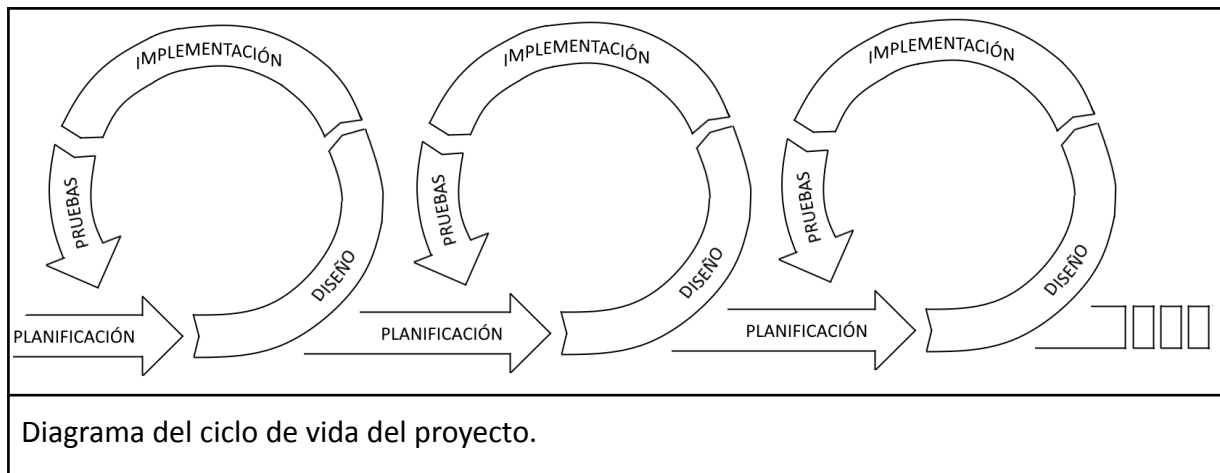
Como ya ha sido postulado, la meta principal es la creación de una herramienta para la gestión y divulgación de información taxonómica. Con este fin, el proyecto se articula en torno a los siguientes objetivos específicos:

- Diseño de una interfaz de usuario, que de una forma intuitiva, permita que los usuarios naveguen de forma eficiente a través de las claves, reduciendo la dependencia del conocimiento especializado en terminología taxonómica.
- Integrar la aplicación con bases de datos relacionales^[4] y no relacionales^[3] para asegurar una gestión eficiente, robusta y flexible de los datos, aprovechando las ventajas de cada paradigma, optimizando el rendimiento y la escalabilidad.
- Desarrollo de una API REST^[1] que actúe como interfaz de comunicación entre el frontend^[10] y el backend^[2], facilitando el intercambio de información en formato JSON. Esta aporta una capa de abstracción sobre el acceso a los datos, el procesamiento de los mismos y la implementación de los casos de uso, dotando de grán escalabilidad a la aplicación debido a la separación de las responsabilidades y al uso de interfaces definidas para los contratos de comunicación^[6].
- Realizar pruebas de aceptación con los usuarios finales para validar la eficacia de la herramienta en diversos entornos y con diferentes grupos de organismos.

3 - Fases del proyecto

Dado que el tiempo para la planificación y desarrollo de este proyecto es muy limitado, para poder gestionarlo de forma eficiente, se ha establecido una metodología de desarrollo ágil, en concreto SCRUM.

El ciclo de vida del proyecto se ha estructurado en distintas fases: planificación, diseño, implementación y pruebas. Aunque en los siguientes apartados se presentan los resultados de estas fases agrupados por sus respectivas categorías, es importante destacar que el trabajo se ha llevado a cabo de forma iterativa. En cada ciclo incremental, se han realizado actividades correspondientes a todas estas fases, posteriormente han sido recopiladas, agrupadas y reflejadas en este documento.



Las ventajas más relevantes de esta metodología de desarrollo en el contexto de mi proyecto son:

- **Desarrollo Incremental:** Permite avanzar de manera progresiva en el desarrollo de la herramienta interactiva para la interpretación de claves taxonómicas, entregando partes funcionales del proyecto en cada sprint. Esto facilita la incorporación de mejoras continuas y ajustes basados en el feedback recibido.
- **Flexibilidad y Adaptación:** La metodología SCRUM se adapta bien a los cambios en los requisitos, lo cual es beneficioso en un proyecto que puede requerir la incorporación de nuevas funciones o ajustes según el feedback de los expertos en entomología.
- **Transparencia y Control:** La utilización de herramientas como JIRA para gestionar SCRUM proporciona una visualización clara del progreso del proyecto, permitiendo un seguimiento detallado de las tareas y facilitando la planificación y organización.

La implementación de esta metodología trae como consecuencia una serie de desventajas:

- **Carga de Trabajo Individual:** Al ser un proyecto llevado a cabo por una sola persona, asumir los roles de una metodología ágil resulta agotador y difícil de manejar, reduciendo la eficiencia y añadiendo carga de trabajo y tiempo extra que hay que dedicar a un proyecto para el cual el margen de tiempo del que se dispone ya es suficientemente corto.
- **Tiempo en Planificación:** Aunque las reuniones de planificación y retrospectiva son esenciales en SCRUM, en este proyecto las reuniones formales se han reemplazado por autoevaluaciones, lo que requiere de una alta disciplina personal.

- **Limitaciones en la Implementación Completa:** Esta metodología está diseñada para equipos, por lo que algunas prácticas, como las reuniones diarias de stand-up y la planificación de sprints en equipo, no se pueden implementar de manera efectiva en un proyecto individual, además, el fin último de alguna de las reuniones es el intercambio de conocimiento entre el equipo de desarrollo para evitar bloqueos frente a problemas que ya ha podido resolver algún otro integrante. En un desarrollo individual, no existe dicha posibilidad.

3.1 - Planificación - (Análisis)

El análisis de los requisitos del software es crucial para definir las necesidades y expectativas, tanto en términos funcionales como no funcionales.

Tras introducir los requisitos de la aplicación, describiendo a grandes rasgos los requisitos de los usuarios de la misma (requisitos funcionales), así mismo como los requisitos técnicos del sistema en su conjunto (requisitos no funcionales), las interacciones posibles con el sistema serán representadas mediante un diagrama de casos de uso.

3.1.1 - Análisis de los requisitos del software

Este proyecto se centra en el desarrollo de una herramienta interactiva para la interpretación de claves taxonómicas, y el análisis detallado de estos requisitos es esencial para asegurar su éxito.

3.1.1.2 - Requisitos funcionales

a. Creación de perfiles de usuario

Los usuarios deben poder autenticarse en la aplicación y acceder exclusivamente a su información o a la que le ha sido compartida.

Casos de Uso:

- ❖ El usuario puede registrarse.
- ❖ El usuario puede hacer login.
- ❖ El usuario puede acceder a su perfil.

b. Creación de Claves Personalizadas

Los usuarios deben poder crear y modificar y compartir sus propias claves taxonómicas, adaptándolas a sus necesidades específicas.

Casos de Uso:

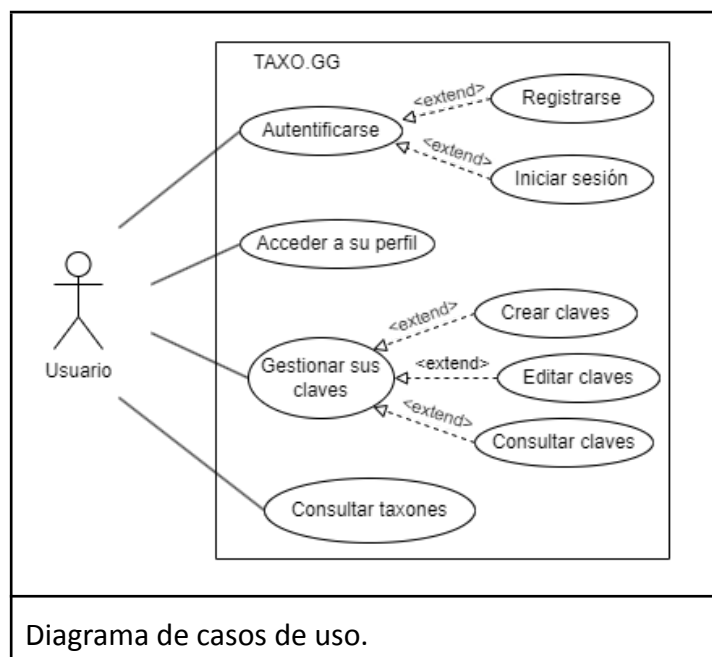
- ❖ El usuario puede iniciar un proceso para crear una nueva clave taxonómica, definiendo las preguntas y respuestas.
- ❖ El usuario puede modificar una clave existente que le pertenezca o que le haya sido compartida.

c. Identificación de Organismos

La aplicación debe permitir a los usuarios identificar organismos a través de una interfaz interactiva que proporciona funcionalidades que asistan en la navegación a través de las claves que ellos mismos han creado o que les han sido compartidas.

Casos de Uso:

- ❖ El usuario puede consultar sus claves taxonómicas.
- ❖ El usuario puede acceder a datos de taxones^[14] y características taxonómicas



3.1.1.2 - Requisitos no funcionales

a. Rendimiento y Escalabilidad:

La aplicación debe ser capaz de manejar múltiples usuarios simultáneamente y responder de manera eficiente. La estructura de datos ha de estar optimizada para usar el menor espacio posible en las bases de datos.

Las consultas más pesadas sobre datos que varíen con poca frecuencia han de ser almacenadas en caché^[5] para evitar bloqueos en las bases de datos o disminución del rendimiento, además serán cacheadas también a nivel de la interfaz gráfica en la sesión del usuario para evitar saturar la interfaz REST con consultas repetitivas por parte del usuario.

b. Seguridad:

Garantizar la seguridad de los datos, incluyendo autenticación y autorización robustas. Las conexiones a las bases de datos no deben ser públicas y deben de hacerse por medio de una interfaz REST que controle el acceso a los datos, evitando inyección. La información sensible ha de estar configurada de forma segura en las variables de entorno de las máquinas de los entornos de ejecución.

c. Usabilidad:

La interfaz de usuario debe ser intuitiva y fácil de usar para personas con diferentes niveles de experiencia en taxonomía. El objetivo es un diseño centrado en el usuario con interfaces claras y generadas dinámicamente en función de la información a la cual tiene acceso el usuario.

d. Mantenibilidad y Extensibilidad:

El software debe ser fácil de mantener y extender para incorporar nuevas funcionalidades y mejoras.

Para asegurar la escalabilidad del backend^[2], el cual contiene la lógica de negocio, la interfaz REST será desarrollada en base a los principios de la arquitectura hexagonal, estándar que dota a cada componente de un grán nivel de abstracción sobre el resto de componentes con los que está conectado, minimizando la complejidad a la hora de hacer modificaciones o de escalar la aplicación.

3.1.2 - Metodología de Desarrollo

Como ya ha sido mencionado, el desarrollo del proyecto ha sido gestionado por medio de una metodología ágil, lo cual ha permitido un desarrollo incremental. El marco de trabajo por medio del cual se ha implementado la metodología ágil ha sido Scrum.

3.1.2.1 - Descripción del marco de trabajo

Scrum es un marco de trabajo para el desarrollo ágil de software. Está basado en la aplicación de un conjunto de buenas prácticas para trabajar de forma colaborativa.

El adoptar esta metodología trae consigo la ventaja de que, al centrarse en ciclos de desarrollo cortos y productivos llamados sprints, dividiendo el desarrollo en pequeñas metas llamadas historias, que ayudan a planificar y organizar el tiempo.

En SCRUM, la dificultad o complejidad de cada historia se mide mediante los puntos de historia. El valor de esta unidad de medida es determinado por el equipo de desarrollo al comienzo del proyecto. La estimación de cuántos puntos vale cada historia también es efectuada por el equipo de desarrollo, pero esta vez en el “sprint planning” previo al comienzo de cada sprint. Estos se asignan siguiendo una versión modificada de la secuencia de Fibonacci 1, 2, 3, 5, 8, 13, 20, 40, y 100.

Las principales desventajas recaen en el tiempo que requiere la planificación y reuniones del equipo y en los posibles problemas de coordinación entre el equipo de desarrollo.

3.1.2.2 - Adaptaciones Específicas

He decidido que cada punto de historia en este proyecto va a tener un valor de 30 minutos, ya que es, de forma aproximada, el tiempo mínimo que creo que me puede llevar una tarea sencilla, como por ejemplo la creación y configuración de los repositorios. Así mismo determiné que la duración de un sprint, de forma que se adaptara a las necesidades de mi situación debía de ser de dos semanas, ya que es un tiempo corto, para obligarme a generar pequeños bloques de contenido de forma continua, pero era lo suficiente largo para que me diese tiempo a desarrollar funcionalidades completas de algún aspecto de la aplicación.

Dado que el desarrollo de este proyecto ha sido llevado a cabo por una sola persona, he adaptado la metodología SCRUM para que se ajuste a las necesidades y limitaciones de trabajar individualmente. En este sentido, he asumido los roles de Product Owner, Scrum Master y equipo de desarrollo. Debido a esta circunstancia, no he realizado reuniones

formales de Sprint Planning, Daily Stand-up ni Sprint Retrospective. En su lugar, he llevado a cabo autoevaluaciones para mantener la disciplina y el control del proyecto.

Como he mencionado anteriormente, la principal desventaja de SCRUM es el tiempo que hay que invertir en planificación y coordinación del equipo, no obstante, dichos problemas se han visto mitigados en mi caso, por el hecho de que el equipo de este proyecto está únicamente conformado por mí, así que los problemas de coordinación desaparecen y porque el tiempo dedicado a la planificación, ha sido recompensado en forma de documentación del proceso de desarrollo, que poder reflejar en esta memoria.

3.1.2.3 - Plataforma para la gestión de SCRUM

La herramienta elegida para gestionar SCRUM durante el desarrollo de este proyecto es JIRA, una herramienta robusta y ampliamente utilizada en el desarrollo de software ágil. Permite una visualización clara del progreso del proyecto, facilitando el seguimiento detallado de las tareas y la planificación organizada.

El backlog del proyecto será gestionado en JIRA, priorizando las características y tareas según su importancia y el tiempo disponible. En JIRA, cada tarea, a grandes rasgos se llama épica, estas se desglosan en historias de usuario, a las cuales se asignan los puntos de historia para estimar la dificultad y el tiempo necesario para completarlas.

3.1.3- Análisis de las alternativas

Antes de comenzar con el diseño de la aplicación, hay que elegir las tecnologías por medio de las cuales esta va a ser construida, ya que estas pueden limitar u ofrecer alternativas sobre ciertos aspectos del desarrollo de la misma.

3.1.3.1 - Tecnologías para el frontend (interfaz de usuario)

Antes de elegir las tecnologías específicas que serán empleadas para el desarrollo de la interfaz de usuario, hay que valorar el tipo de aplicación, ya que esto determinará los lenguajes y frameworks entre los cuales podemos elegir.

Una aplicación nativa trae consigo algunas ventajas, entre ellas, estas aplicaciones son las que mejor rendimiento ofrecen, no obstante, para que pueda ser multiplataforma ha de ser programada en lenguajes o frameworks muy concretos, que aún así, no siempre se adaptan por completo a todos los dispositivos y/o sistemas operativos, dando lugar a la necesidad de desarrollos individualizados para cada plataforma.

Una aplicación web es completamente multiplataforma, ya que esta es interpretada, no necesita ser compilada ni instalada. Además, una aplicación web es alojada en un servidor web, permitiendo el acceso desde cualquier navegador, lo que la hace accesible desde cualquier equipo. No obstante, esto trae consigo la necesidad de contar con conexión a internet. En este caso en concreto, esto no es un inconveniente, ya que el ámbito de uso de la aplicación es la investigación y el estudio de los taxones^[14], tarea que es llevada a cabo de forma general en universidades o en laboratorios, instalaciones que por norma general, cuentan con equipos con acceso a la red.

Valorado todo esto, considero que para que la aplicación sea completamente accesible desde cualquier equipo, es esencial recurrir a tecnologías web. Con todo esto en mente, ahora se han de decidir las tecnologías web que serán empleadas para el desarrollo de la misma.

La creación de claves taxonómicas requiere de una interfaz de usuario muy flexible, que permita generar estructuras completas de forma dinámica, para lo cual considero que requiere de un lenguaje que permita la programación funcional. Con esto ya en mente, tengo claro que he de optar por JavaScript, no obstante, existen diferentes frameworks de este lenguaje que hay que valorar.

React es una de las librerías más populares y utilizadas para el desarrollo de interfaces de usuario. Además, la programación funcional que permite React con el uso de hooks es muy potente. Sin embargo, React introduce una capa adicional de complejidad con JSX y la necesidad de transpilación^[15] con herramientas como Babel, además tiene dependencia con otros frameworks que harían la aplicación mucho más pesada.

Angular es un framework completo que proporciona una solución integral para el desarrollo de aplicaciones web. No obstante, su curva de aprendizaje es pronunciada y puede ser excesiva para aplicaciones que no requieren tanta estructura y lo hacen inviable para un desarrollo express en el cual se dispone de muy poco tiempo.

Dado que la aplicación que se va a desarrollar necesita una interfaz flexible y dinámica pero no tiene una complejidad extremadamente alta, optar por JavaScript puro ejecutado directamente en el navegador del usuario es la opción que mejor se adapta a mis necesidades. Usar JavaScript nativo evita la necesidad de aprender y configurar frameworks adicionales. Esto reduce la complejidad del proyecto y facilita el mantenimiento a largo plazo. Además al no depender de frameworks pesados, el rendimiento es superior y no tengo que preocuparme por que estos sean discontinuados o se queden sin soporte, evitando la dependencia de terceros. JavaScript es soportado nativamente por todos los navegadores modernos, lo que garantiza una amplia compatibilidad sin necesidad de configuraciones adicionales. Por último, trabajar con JavaScript puro proporciona un control total sobre el código y la estructura de la aplicación, lo que permite optimizar y personalizar cada aspecto según las necesidades específicas de mi aplicación.

Con todo esto en mente, creo que el lenguaje para el desarrollo de la interfaz de usuario que mejor se adapta a las necesidades de mi proyecto es JavaScript nativo.

3.1.3.2 - Tecnologías para el acceso a los datos

La gestión del acceso a los datos se puede realizar por medio de diferentes enfoques, desde la serialización de la información en ficheros locales, uso de bases de datos locales, servidores de bases de datos online o interfaces REST.

Para satisfacer los requisitos de mi aplicación, podemos descartar de forma directa el uso de recursos locales, ya que la información ha de ser centralizada para que pueda estar actualizada y para que sea accesible desde cualquier parte y compartida entre los usuarios.

Dicho esto, está claro que he de usar una base de datos online, no obstante, aplicar esto de forma directa, conlleva muchos riesgos, ya que exponer directamente una base de datos a internet la deja desprotegida frente a ataques de inyección de código, fuerza bruta, etc...

Por esto mismo, para el backend^[2] de la aplicación, he decidido crear una API REST^[1], como interfaz de comunicación entre el frontend^[10] de la aplicación (interfaz de usuario) y el backend^[2] (la lógica de negocio y acceso a datos) de la misma. Esta separación de responsabilidades cuenta con las siguientes ventajas:

- Centralización del acceso a datos simplificando la gestión de las conexiones, facilitando la gestión y mantenimiento de la lógica de datos sin que los cambios afecten al frontend^[10].
- Incremento en la escalabilidad y simplificación del mantenimiento de cada componente de forma aislada, derivada de la separación en diferentes capas (presentación, lógica de negocio y acceso a datos).
- Seguridad por ocultación de las credenciales de acceso a la base de datos. Al ser almacenadas en el servidor de la api, las credenciales no son expuestas al cliente.

3.1.3.3 - Tecnologías de la interfaz REST

Teniendo claro que es necesaria una API REST^[1] para gestionar el acceso a los datos de forma segura, para elegir la tecnología en la cual se desarrollará la interfaz REST, tendré en cuenta algunas de las principales tecnologías disponibles para la creación de APIs.

Node.js es una plataforma de desarrollo que utiliza JavaScript en el lado del servidor, y Express es un framework minimalista para Node.js que se usa ampliamente para desarrollar APIs REST. Me permitiría escribir tanto el frontend^[10] como el backend^[2] en JavaScript, lo que

simplificará el desarrollo y reduciría la curva de aprendizaje. Node.js es una tecnología nueva y puede no ser tan estable y maduro como algunas soluciones basadas en lenguajes más antiguos. Las aplicaciones Node.js pueden consumir más memoria que otras soluciones basadas en PHP o Python.

Django es un framework de desarrollo web de alto nivel en Python, y el Django REST Framework es una herramienta flexible para construir APIs web. Django tiene muchas características de seguridad incorporadas, como protección contra inyecciones SQL y CSRF. No obstante, Django y DRF pueden tener una curva de aprendizaje pronunciada, especialmente para desarrolladores no familiarizados con Python. Además, Django es muy pesado para proyectos que no requieren todas sus funcionalidades integradas.

Slim PHP es un micro framework para PHP que se centra en ser ligero y sencillo de usar, ideal para la creación de APIs RESTful. Es extremadamente ligero, lo que se traduce en un rendimiento rápido y eficiente. Permite agregar solo las características necesarias, evitando la sobrecarga de funcionalidades innecesarias. Además tiene una comunidad muy activa, lo cual facilita el proceso de buscar soluciones frente a errores que puedan surgir en el desarrollo. Aunque tendría que aprender PHP, hay mucho contenido disponible en internet y no es considerado un lenguaje complejo.

Habiendo valorado las diferentes alternativas de las que dispongo, creo que el lenguaje para el desarrollo de APIs REST que mejor se adapta a las necesidades de mi proyecto es PHP, por medio del framework Slim.

3.1.3.4 - Tecnologías para la base de datos

Para la elección de las tecnologías a emplear para implementar las bases de datos, hay que tener muy presente la naturaleza de los propios datos que serán almacenados en las mismas.

La aplicación debe gestionar la información de los usuarios y permisos, estos datos tienen una estructura definida, no variable de fácil implementación en un sistema relacional. Además, la aplicación debe ser capaz de almacenar datos de taxones^[14] y de claves taxonómicas, de naturaleza muy flexible y variable, estos datos requieren de un sistema muy flexible en el cual no se requiera de declarar el tamaño de los campos, para evitar desperdiciar espacio en la base de datos.

Teniendo en cuenta las valoraciones anteriores, la aplicación debe integrarse con bases de datos relacionales^[4], para gestionar la información de los usuarios y permisos y no relacionales para almacenar y gestionar datos taxonómicos.

Para la base de datos relacional^[4], he tenido en cuenta el posible uso de MySQL, PostgreSQL y MariaDB.

MySQL es ampliamente utilizado, veloz y fiable. Sin embargo, tiene limitaciones en cuanto a la flexibilidad de las licencias y algunas funcionalidades avanzadas que están restringidas en la versión comunitaria.

PostgreSQL es muy robusto y con características avanzadas como soporte para tipos de datos personalizados, pero es más complejo de administrar y no tan ligero en términos de rendimiento para aplicaciones pequeñas.

MariaDB, una bifurcación de MySQL, combina la velocidad y simplicidad de MySQL con mejoras en el rendimiento y características adicionales, además de ser completamente open source, lo cual elimina las preocupaciones sobre licencias.

Para la base de datos no relacional^[3], he tenido en cuenta CouchDB, Cassandra y MongoDB.

CouchDB es adecuado para aplicaciones que requieren una alta disponibilidad y facilidad de replicación, con un modelo de datos basado en documentos y una API REST^[1], pero puede no ser tan eficiente en operaciones de búsqueda complejas.

Cassandra ofrece una escalabilidad masiva, ideal para manejar grandes volúmenes de datos a través de múltiples servidores, aunque su modelo de consistencia eventual puede ser un desafío para ciertas aplicaciones que requieren transacciones estrictas.

MongoDB es altamente flexible y permite un almacenamiento eficiente de datos semi-estructurados con consultas potentes y soporte para operaciones complejas, además al haber trabajado anteriormente con esta tecnología, no tendría que invertir tiempo en aprender cómo se usa.

Después de evaluar estas opciones, he decidido utilizar MariaDB como base de datos relacional^[4] y MongoDB como base de datos no relacional^[3].

3.1.4 - Análisis de costos y beneficios

Tras definir los objetivos y las tecnologías a emplear, para realizar una evaluación del impacto de este proyecto, hay que analizar tanto los costos de implantación del sistema como los beneficios que puede aportar el mismo.

3.1.4.1 - Costos

El costo principal de este proyecto es la inversión en tiempo requerido para la planificación, implementación y pruebas del mismo. El desarrollo del proyecto no trae consigo costos económicos ya que todas las tecnologías elegidas para el desarrollo del mismo son tecnologías libres o permiten su uso de forma gratuita. Así mismo los entornos de ejecución de los componentes de la aplicación serán emulados mediante contenedores de Docker^[8] en lugar de alquilar servidores reales.

3.1.4.2 - Beneficios

Como ya ha sido expresado en la introducción, uno de los principales beneficios de la implantación de esta aplicación, vendría en forma de apoyo a la investigación en todo el ámbito de la identificación de organismos vivos, ya que bien planteada y pensada para que pueda cubrir el espectro completo de organismos, la plataforma servirá para elaborar e interpretar claves taxonómicas personalizadas que se centren en el grupo taxonómico de elección del usuario.

La herramienta reducirá el tiempo de consulta de estas claves por medio de una interfaz amigable que asista en la navegación a través de las mismas, haciendo así también más sencilla la introducción al mundo de la taxonomía a usuarios sin experiencia en dicho campo.

3.2 - Diseño

Tras la elección de las tecnologías en las que será construida la aplicación, en este apartado introduciré los componentes de software que conforman las diferentes capas de la misma, describiendo el rol que cumplirán dentro de esta, y los aspectos que he priorizado en el diseño de los mismos.

3.2.1 - Interfaz de usuario

La interfaz de usuario ha sido conceptualizada como una aplicación web para asegurar su accesibilidad y facilidad de uso y flexibilidad.

El diseño de los “Mocks” de la interfaz gráfica ha sido reflejado en los apéndices.

3.2.1.1 - Descripción de la interfaz (capa de presentación)

La prioridad es que sea una estructura intuitiva que permita a los usuarios navegar de manera eficiente a través de los taxones^[14] y las claves taxonómicas, minimizando la necesidad de conocimientos especializados. Además, en el diseño de la interfaz gráfica se han priorizado los siguientes aspectos:

- **Estructura de Navegación:** La interfaz ha de presentar una estructura jerárquica clara, con un menú que guíe al usuario a través de las diferentes funcionalidades, estas funcionalidades han de estar asociadas a un sistema de pestañas interno de la página, esto representa un rendimiento superior y una gestión más eficiente de la memoria del dispositivo frente a la apertura de cada sección en una pestaña nueva del navegador.
- **Interactividad:** Se incluyen elementos interactivos como botones y formularios para facilitar la interacción del usuario con la aplicación.
- **Accesibilidad:** La interfaz está diseñada para ser accesible desde cualquier dispositivo con conexión a internet, garantizando un acceso amplio y sin restricciones.

3.2.2 - API REST

La API REST^[1] actúa como intermediaria entre el frontend^[10] y los servidores de las bases de datos, proporcionando una capa de abstracción y seguridad, además de contener la lógica de negocio de la aplicación.

3.2.2.1 - Descripción de la API REST (capa de lógica de Negocio)

La API se ha diseñado para manejar de manera eficiente las solicitudes del usuario y garantizar la integridad y seguridad de los datos. En el diseño de la interfaz REST se han tenido en cuenta los siguientes aspectos:

- Reducción de la carga en base de datos por medio de respuestas en caché^[5] para las peticiones más habituales, mejorando también la experiencia de usuario al acelerar las respuestas.
- Seguridad activa mediante la implementación de capas de autenticación y autorización a nivel de api (el usuario no puede visualizar ni manipular el código de la misma).

3.2.2.2 - Diagrama de clases

El siguiente diagrama de clases muestra la estructura de la lógica de negocio de la API REST^[1], reflejando las entidades principales y sus relaciones:

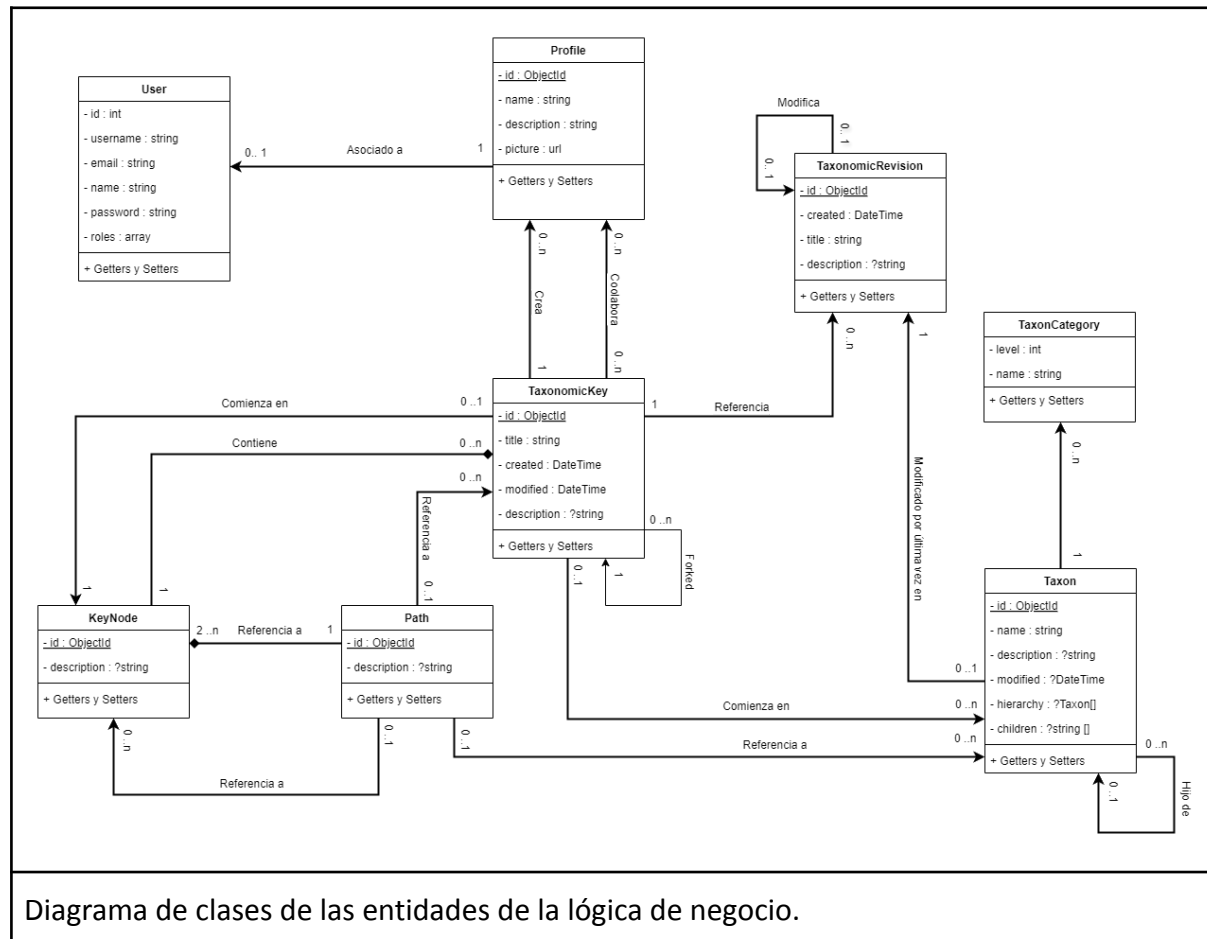


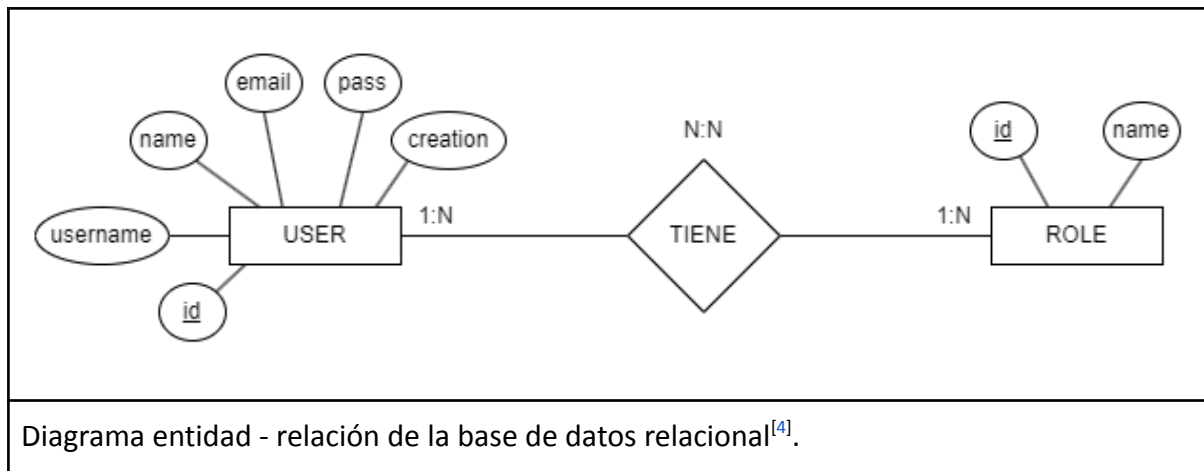
Diagrama de clases de las entidades de la lógica de negocio.

3.2.3 - Bases de datos

Resumiendo lo ya definido en la fase de análisis, dada la naturaleza diversa de los datos que requieren de ser persistidos y gestionados, se ha adoptado un enfoque versátil que combina una base de datos relacional^[4] para datos altamente estandarizados y referenciales (MariaDB) y una base de datos no relacional^[3] para datos flexibles y variables (MongoDB).

3.2.3.1 Bases de datos relacionales

MariaDB es fundamental en el entorno de la aplicación, será empleada para almacenar y facilitar el acceso de forma eficiente a la información relacionada con los usuarios, sus grupos a los cuales les serán concedidos permisos por medio de la API REST^[1].



Para transformar el diagrama en una estructura almacenable en MariaDB, hay que transformar las entidades y relaciones a un conjunto de tablas, dado el diagrama que he planteado, considero que se requieren las siguientes tablas:

1. Tabla para la información de los usuarios.
User (id, username, name, email, pass, creation)
2. Tabla para los roles.
Role (id, name)
3. Tabla que asocie qué usuarios tienen qué roles, expresado en el diagrama por medio de la relación "TIENE".
User_Role (↑ id_user, ↑ id_role)

Para comprobar que mi planteamiento inicial es correcto hay que realizar la normalización de las tablas, reflejada en los apéndices del proyecto.

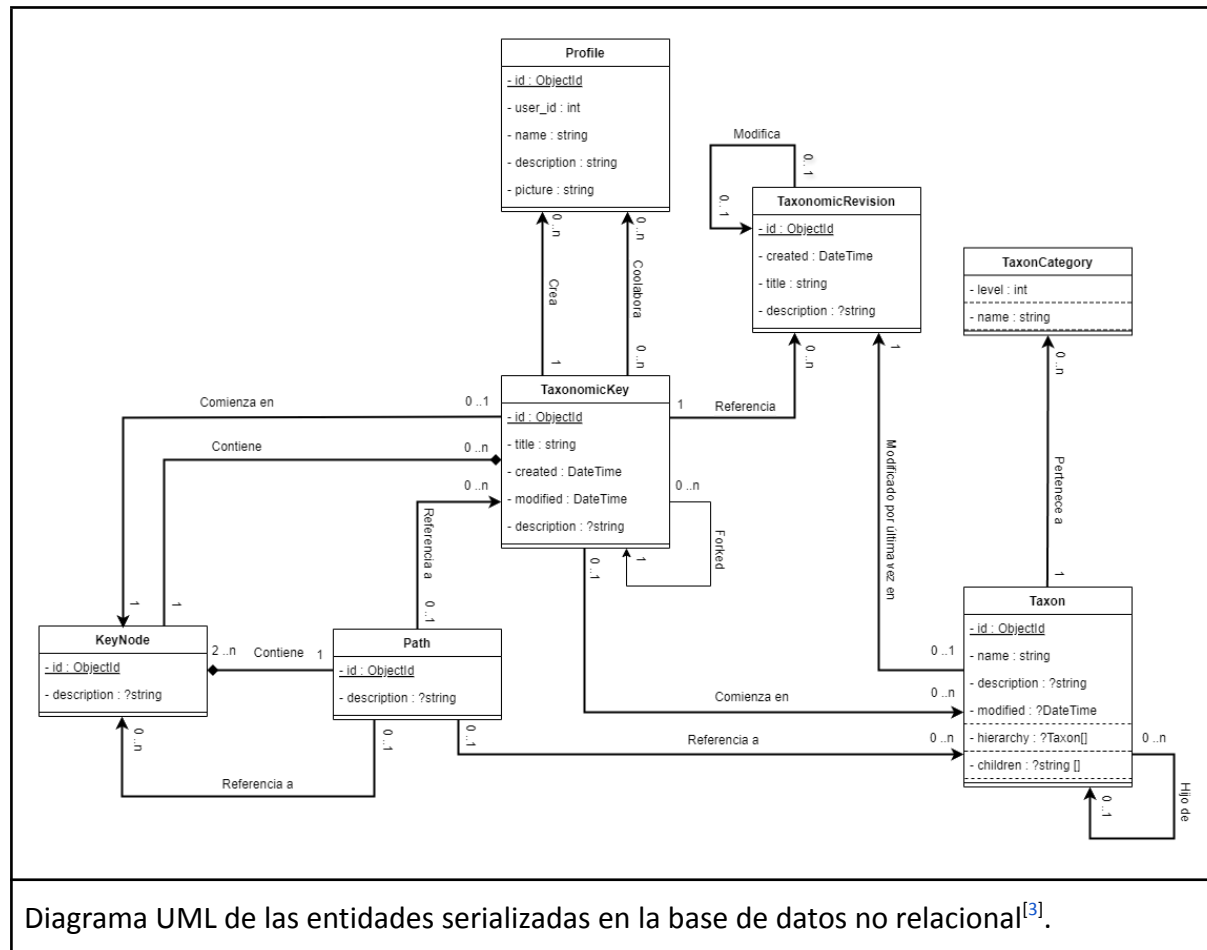
3.2.3.2 Bases de datos no relacionales

Tras determinar en la fase de análisis que las características intrínsecas de MongoDB responden de manera adecuada a la necesidades de la aplicación, tales como almacenar, gestionar y dar acceso a estructuras de datos de naturaleza variable y por otras características de esta base de datos como la velocidad y escalabilidad de la misma, se ha integrado MongoDB en el ecosistema de la aplicación.

Es complejo transmitir de forma abreviada una explicación concisa del por qué elegí la estructura en la que decidí almacenar los datos, esta misma es fruto de varias iteraciones de análisis de los requisitos, elaboración de diagramas, implementación, pruebas de eficiencia, reconsideración y vuelta a empezar. Así sucesivamente hasta que encontré una estructura, que junto con la implementación de una lógica de consultas que almacena las respuestas

temporalmente en caché^[5], consigue optimizar al máximo el espacio en la base de datos, reduciendo a nulo los datos repetidos y a su vez no penalizar en rendimiento debido a la complejidad de las consultas gracias a las respuestas almacenadas en caché^[5].

El siguiente diagrama UML representa las entidades que son serializadas en MongoDB, así como las relaciones de composición, agregación y asociación entre las mismas.



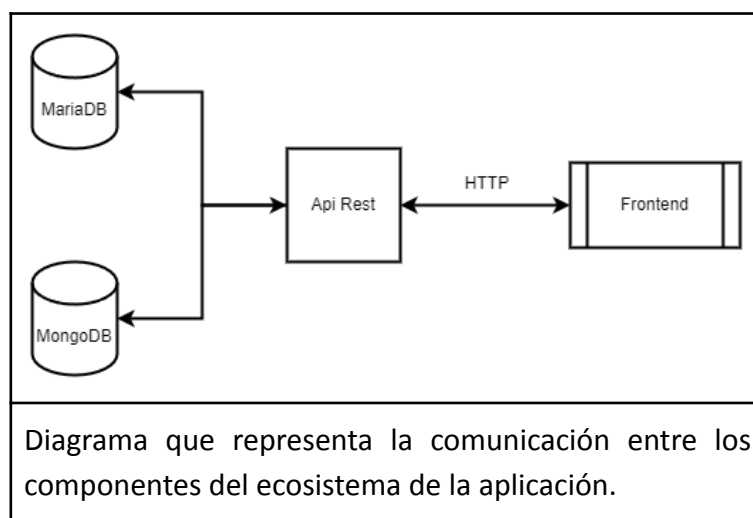
En el diagrama, los campos que actúan como claves foráneas de otras entidades se representan mediante relaciones de asociación. Por otro lado, los campos que son tipos de otras entidades (Contienen dentro un objeto completo de esa entidad) se representan a través de relaciones de composición. Los campos rodeados por líneas discontinuas son calculados por la API REST^[1] mediante consultas recursivas y no se almacenan en la base de datos.

3.2.5 - Arquitectura de hardware

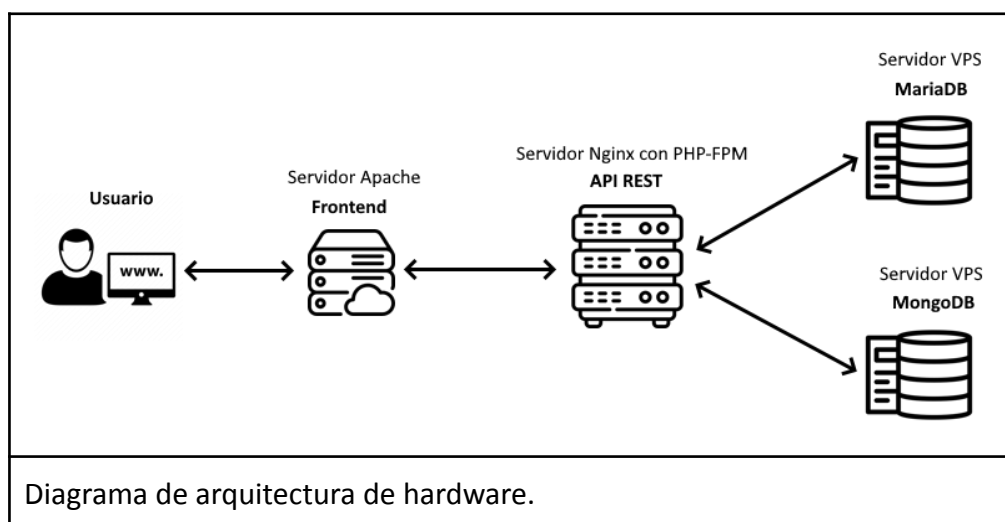
En la fase de análisis, fué determinado que, para tratar de maximizar el rendimiento y la escalabilidad del ecosistema de la aplicación así como por cuestiones de seguridad, era necesaria la fragmentación de la arquitectura de la aplicación en varios sistemas (interfaz de

usuario, API REST^[1], base de datos relacional^[4] y base de datos no relacional) distribuyendo y aislando las diferentes partes ya mencionadas en función de sus responsabilidades.

El entorno de desarrollo de la aplicación es un dispositivo portátil con un sistema operativo Windows 10. Al no corresponder con los sistemas en los cuales está preconcebida la implementación de la aplicación en producción, los diferentes componentes del entorno de la aplicación han sido configurados para ser ejecutados en contenedores Docker^[8], que en conjunto, están programados para que se comuniquen entre ellos, simulando el ecosistema presentado en el siguiente diagrama.



En producción estos contenedores serían reemplazados por servidores linux que, interconectados entre sí ofrecerían un entorno más robusto y escalable. La transición implicaría asegurar la compatibilidad de configuraciones y scripts, configurar adecuadamente la red y la persistencia de datos y automatizar el despliegue mediante herramientas de CI/CD (Integración Continua y Despliegue Continuo) para asegurar la escalabilidad de la aplicación.



3.2.6 - Arquitectura de software

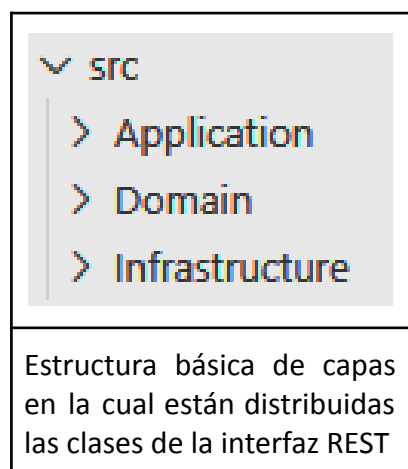
El ecosistema de la aplicación se ha dividido en diferentes componentes en los cuales han sido distribuídas las responsabilidades, que en conjunto, dotan de funcionalidad al sistema:

- **Frontend**^[10]: Proporciona la interfaz de usuario y maneja la interacción con el usuario.
- **API REST**^[1]: Facilita la comunicación segura entre el frontend^[10] y las bases de datos gestionando las sesiones de los usuarios.
- **Bases de Datos**: Gestionan y almacenan la información de usuarios y datos taxonómicos.

El modelo de arquitectura más relevante es probablemente el de la API REST^[1], componente que encapsula toda la lógica de negocio de la aplicación, con lo cual ha de ser muy limpia y escalable.

Para lograr estos objetivos, he decidido emplear la **Arquitectura Hexagonal**, también conocida como Arquitectura de Puertos y Adaptadores, facilita la separación de las responsabilidades y permite que la aplicación sea fácilmente extensible y adaptable a futuros cambios.

Esta arquitectura, define la separación de las clases de la aplicación en tres capas diferentes, las capas de aplicación, dominio e infraestructura:



- **Aplicación:** Contiene la lógica de aplicación, incluyendo los casos de uso y servicios de aplicación, orquestando las interacciones entre el dominio y la infraestructura.
- **Dominio:** Contiene las entidades, objetos de valor y agregados que representan el núcleo de la lógica de negocio.
- **Infraestructura:** Proporciona la implementación técnica necesaria para interactuar con sistemas externos, como bases de datos, APIs, y servicios externos.

3.2.7 - Patrones de diseño

➤ Singleton:

```
class Router {  
    static instance;  
  
    url;  
    params;  
    currentParameters;  
  
    constructor() {  
    }  
  
    static getInstance(){  
        if(Router.instance == null){  
            Router.instance = new Router();  
            Router.instance.initParams();  
            Router.instance.init();  
        }  
        return Router.instance;  
    }  
}
```

Patrón singleton en la interfaz gráfica.

El patrón Singleton asegura que una clase tenga una única instancia y proporciona un punto de acceso global a esa instancia. En el contexto de este proyecto, este patrón se ha utilizado en la interfaz gráfica para gestionar algunas clases como el router, que maneja la navegación entre las diferentes pestañas. Este patrón garantiza que solo exista una instancia del router en toda la aplicación, evitando inconsistencias en la navegación.

Se accede de forma estática a la instancia, si no está iniciada, la inicia (la primera vez), el resto de veces que se llame a este método, se devolverá la instancia ya creada.

➤ Inyección de dependencias

El patrón de Inyección de Dependencias^[11] (Dependency Injection) es un patrón de diseño que permite a los objetos recibir sus dependencias desde una entidad externa en lugar de crearlas por sí mismos. Esto facilita la gestión de dependencias y promueve un código más limpio y modular. En este proyecto, se ha utilizado la inyección de dependencias^[11] para gestionar los servicios y repositorios necesarios para el funcionamiento de la API REST^[1].

En la inyección de dependencias^[11] se declara cómo se instancian las clases a inyectar al ser requeridas en el constructor de otra clase. Al hacer esto, nos despreocupamos de tener que instanciar esa clase en el resto del código, centralizando la gestión de dependencias y mejorando la mantenibilidad del software.

```
$containerBuilder->addDefinitions([  
    MongoControllerInterface::class => function (ContainerInterface $c) {  
        $settings = $c->get(SettingsInterface::class);  
        $dbName = $settings->get(SettingKey::MONGO_DB_NAME);  
        $mongoClient = $c->get(MongoClient::class);  
        return new MongoController($mongoClient, $dbName);  
    },  
]);
```

Inyección de una dependencia^[11] en la API REST^[1].

3.2.8 - Esquema funcional

3.2.8.1 Componentes Principales

1. Interfaz de Usuario (Frontend^[10]): Es la capa de presentación que interactúa directamente con el usuario. Está desarrollada en JavaScript nativo, HTML y CSS.

Funciones:

- a. Mostrar la interfaz gráfica al usuario.
- b. Capturar las acciones del usuario (clics, entradas de datos, etc.).
- c. Enviar solicitudes a la API REST^[1] y mostrar los resultados.

2. API REST^[1] (Backend^[2]): Actúa como intermediario entre la interfaz de usuario y las bases de datos. Está desarrollada en PHP usando el framework Slim.

Funciones:

- a. Recibir y procesar solicitudes HTTP del frontend^[10].
- b. Implementar la lógica de negocio.
- c. Interactuar con las bases de datos para realizar operaciones CRUD.
- d. Devolver respuestas al frontend^[10] en formato JSON.

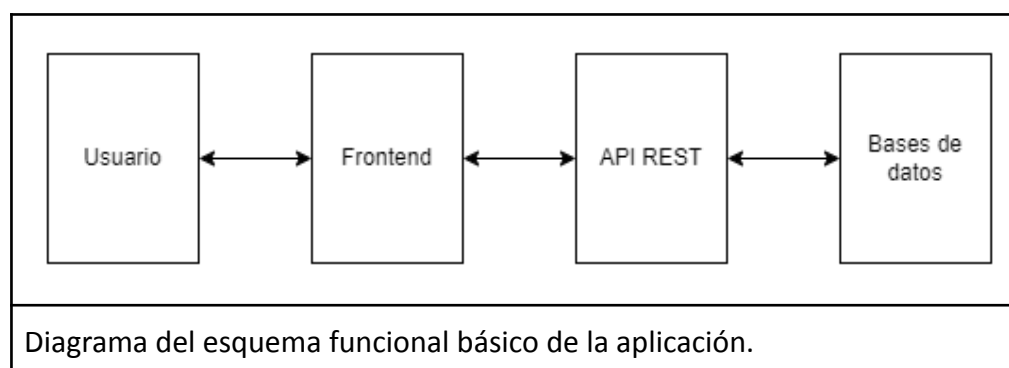
3. Bases de Datos: La base de datos relacional^[4] (MariaDB) gestiona la información de usuarios y roles, la base de datos no relacional^[3] (MongoDB) almacena datos taxonómicos y claves taxonómicas.

Funciones:

- a. Almacenar y recuperar datos de manera eficiente.
- b. Garantizar la consistencia e integridad de los datos.

3.2.8.2 Diagrama del Esquema Funcional

A continuación se presenta un diagrama simplificado que ilustra la comunicación bidireccional entre los componentes del sistema:



Descripción del Diagrama:

- **Usuario a Frontend^[10]**: Los usuarios interactúan con la interfaz gráfica presentada en sus navegadores web.
- **Frontend^[10] a API REST^[1]**: La interfaz de usuario envía solicitudes a la API REST^[1] para realizar diversas operaciones (autenticación, creación de claves, identificación, etc.).
- **API REST^[1] a Bases de Datos**: La API REST^[1] consulta o actualiza la información en las bases de datos según sea necesario.
- **Bases de Datos a API REST^[1]**: Las bases de datos responden a las solicitudes de la API REST^[1] proporcionando los datos necesarios.
- **API REST^[1] a Frontend^[10]**: La API REST^[1] devuelve los resultados de las operaciones al frontend.
- **Frontend^[10] a Usuario**: El frontend^[10] muestra los resultados y actualiza la interfaz según las acciones del usuario.

3.2.8.3 Ejemplos del flujo de trabajo

1. Inicio de Sesión del Usuario

- a. El usuario accede al formulario de inicio de sesión desde su navegador.
- b. El frontend^[10] captura las credenciales del usuario y las envía a la API REST^[1].
- c. La API REST^[1] valida las credenciales consultando la base de datos relacional^[4].
- d. Si las credenciales son correctas, la API REST^[1] genera un token JWT^[12] y lo devuelve al frontend^[10].
- e. El frontend^[10] almacena el token^[12] y redirige al usuario a la página de su perfil.

2. Creación de Claves Taxonómicas

- a. El usuario accede a la interfaz de creación de claves.
- b. El frontend^[10] envía los datos de la nueva clave a la API REST^[1].
- c. La API REST^[1] almacena los datos en la base de datos no relacional^[3].
- d. La API REST^[1] devuelve una confirmación al frontend^[10].

3. Identificación de Organismos

- a. El usuario selecciona una clave taxonómica para identificar un organismo.
- b. El frontend^[10] envía la selección a la API REST^[1].
- c. La API REST^[1] consulta MongoDB para obtener la clave taxonómica correspondiente.
- d. La API REST^[1] procesa la identificación según la clave y devuelve los resultados al frontend^[10].
- e. El frontend^[10] muestra los resultados al usuario.

3.3 - Implementación y configuración

3.3.1 - Gestión de las cargas de trabajo a través del proyecto

Como ya ha sido mencionado, en la fase de planificación se decidió gestionar el proyecto por medio del marco de trabajo de SCRUM, a través de Jira.

El volumen de trabajo total ha sido dividido en bloques y agendado de forma distribuída a lo largo de tres sprints. No obstante, previo a la implementación de dicha metodología, se dió la fase de análisis, antes de llegar a adoptar la dinámica de trabajo por medio de sprints, con lo cual esta fase no se vé reflejada en los mismos, fueron unas 40 horas.

En los siguientes apartados, se destacan las tareas más relevantes llevadas a cabo en cada uno de los sprints:

3.3.1.1 - Primer sprint (16/04/2024 - /30/04/2024) => 70 horas

- Configurar Jira para gestionar el proyecto mediante SCRUM
- Crear y configurar los repositorios de frontend^[10] y backend^[2]
- Creación de los diagramas UML y Entidad-Relación para las bases de datos
- Configurar los contenedores de las bases de datos y los scripts de sql y javascript para generar la estructura de las bases de datos
- Configurar el contenedor para la API REST^[1]

3.3.1.2 - Segundo sprint (1/05/2024 - 14/05/2024) => 60 horas

- Crear sistema de autenticación para la API REST^[1]
- Crear endpoints^[9] para registrarse e iniciar sesión
- Configurar endpoints^[9] par consultar la información de taxones^[14]
- Comenzar a programar componentes de la interfaz gráfica como el menú, el gestor de pestañas, el gestor de idiomas...
- Crear formularios de registro e inicio de sesión y pestaña de “Inicio”

3.3.1.3 - Tercer sprint (15/05/2024 - 29/05/2024) => 80 horas

- Crear endpoints^[9] para crear y modificar claves taxonómicas
- Crear endpoints^[9] para recuperar la información de claves taxonómicas
- Programar la pestaña de búsqueda de taxones^[14]
- Programar la pestaña del listado de claves
- Programar la pestaña de consulta y edición de claves

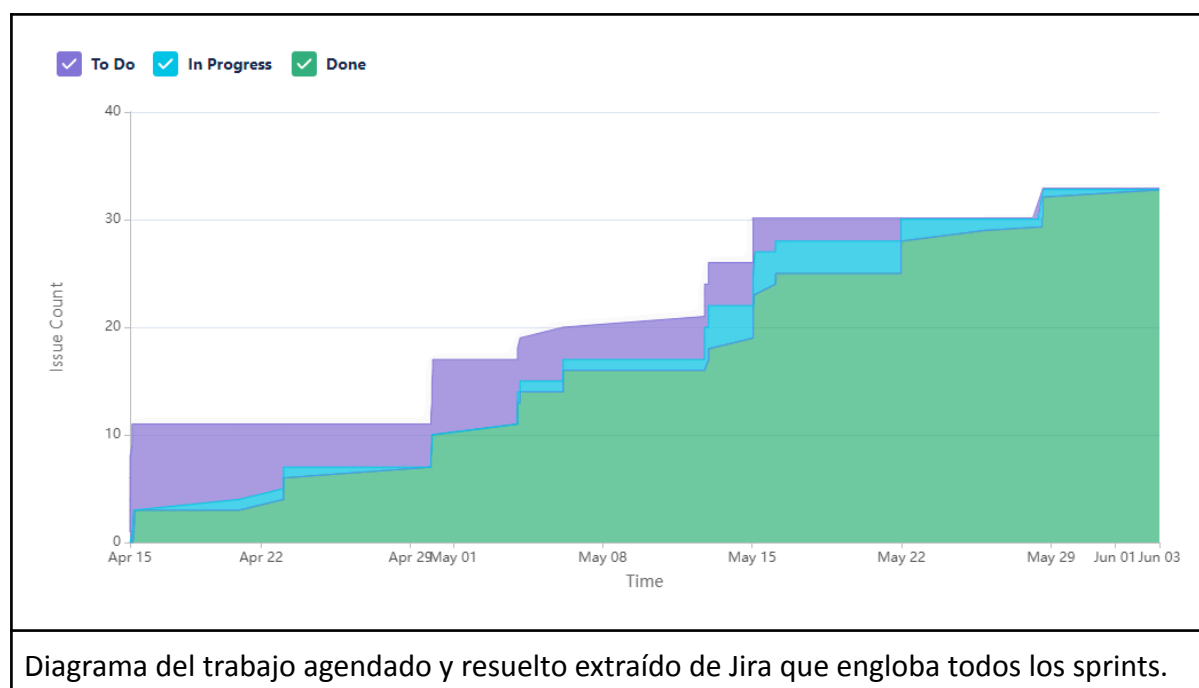
3.3.1.4 - Otras tareas

Tareas como la investigación, documentación o la realización de pruebas han sido paralelas a todo el desarrollo del proyecto, con lo cual no han sido asignadas a ningún sprint o historia específicos, ya que cada una de ellas implicaba buscar información, probar el software de una manera u otra y registrar los resultados en la memoria.

3.3.1.5 - Vista general del proyecto

Jira facilita un diagrama del progreso del proyecto, en el cual se puede apreciar cómo se han ido gestionando las cargas de trabajo a lo largo del desarrollo del proyecto. A lo largo de los sprints las historias por hacer se han ido informando como “To Do”, indicado en el siguiente diagrama en morado, al finalizar las tareas, estas se cerraban y se informaban como terminadas o “Done”, indicado en el diagrama en color verde.

El total de tiempo dedicado al proyecto acumula unas 250 horas, entre investigación, planificación, desarrollo, pruebas, corrección de errores y documentación.



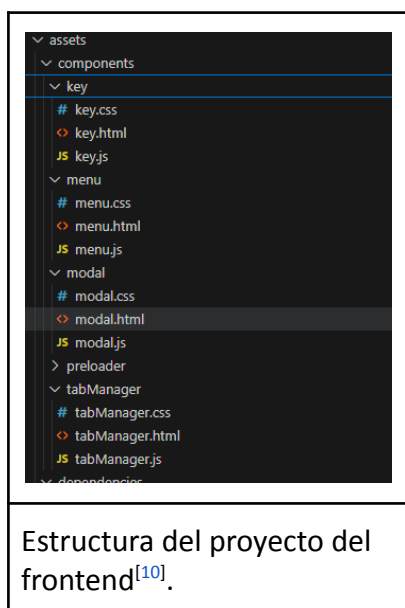
3.3.2 - Interfaz gráfica

Como ya ha sido desarrollado en las fases de análisis y diseño, la interfaz ha sido implementada por medio de una aplicación web escrita en JavaScript nativo, con HTML y CSS. Esta aplicación se ha de ejecutar en un servidor web. Para el desarrollo se ha ejecutado a través de xampp, que emula dicho servidor web en mi equipo personal.

- HTML se utiliza para estructurar el contenido de la web.
- CSS se emplea para estilizar la presentación visual.
- JavaScript gestiona la lógica de la interfaz, permitiendo la interacción dinámica del usuario.

La aplicación se ejecuta en un servidor web, desarrollado usando XAMPP durante el entorno de desarrollo.

Estructura del Proyecto



El proyecto se ha organizado en una estructura de directorios clara, con separaciones para los archivos HTML, CSS y JavaScript de cada uno de los contextos de la aplicación, como el menú, el router, el gestor de pestañas... Esto facilita el mantenimiento y la escalabilidad de la aplicación.

En el contexto de “components” he agrupado las clases que generan elementos gráficos como las clases “Menu” o “Modal” que, además de Javascript tienen o generan html y css.

En el contexto de “dependencies” he agrupado las clases que solo gestionan lógica mediante javascript como las clases “Router”, “Cache”,

3.3.3 - API REST

La API REST^[1] se desarrolló utilizando PHP y el framework Slim, siguiendo los principios de la arquitectura hexagonal para asegurar una separación clara de las responsabilidades y facilitar la escalabilidad y mantenibilidad del código .

Configuración del entorno

Se configuró un contenedor PHP utilizando Docker^[8] para garantizar un entorno de desarrollo consistente. La imagen base de PHP se amplió para incluir todas las dependencias necesarias, como las extensiones PHP y el driver de MongoDB.

Manejo de Sesiones

El manejo de sesiones se implementó utilizando JSON Web Tokens^[12] (JWT). Los tokens^[12] se generan al momento de la autenticación y se verifican en cada solicitud para asegurar que el usuario está autorizado para acceder a los recursos solicitados. Dentro de estos tokens^[12] está encriptada la identificación del usuario y el tiempo de caducidad. Al recibir el token^[12] en una petición, se identifica al usuario y se comprueba mediante una consulta a la base de datos se recuperan los roles a los que pertenece el usuario, entonces, a nivel de api se evalúa si esos roles tienen permiso para acceder a ese endpoint^[9].

Estructura del Proyecto



Estructura básica de las carpetas principales del proyecto de la API REST.

Para entender cómo está distribuido el código en esta aplicación:

- El directorio (**app/**) contiene las configuraciones para la inyección de dependencias^[11], middleware, rutas y repositorios.
 - El directorio (**logs/**) contiene los archivos de log de la aplicación.
 - El directorio (**public/**) contiene el archivo .htaccess, que se encarga de la reescritura de URLs y otras directivas de configuración base de la aplicación, el archivo index.php, configurado como único punto de entrada independientemente de la URL (regla definida en .htaccess), en este archivo (index.php) se crea un contenedor PHP-DI para la inyección de dependencias^[11], a través este mismo, se cargan las configuraciones, las dependencias, y los repositorios. Tras esto, con el contenedor listo, se instancian la app y los manejadores de errores, se cargan los “middlewares”, se redirecciona la solicitud para que sea procesada por la app y se emite la respuesta.
- Dentro del directorio (**src/**) se encuentra toda la lógica de negocio de la aplicación (entidades, casos de uso, repositorios), todas las clases de la misma están organizadas siguiendo el patrón de la arquitectura hexagonal, distribuidas en los tres contextos (aplicación, dominio e infraestructura) en función de su responsabilidad.

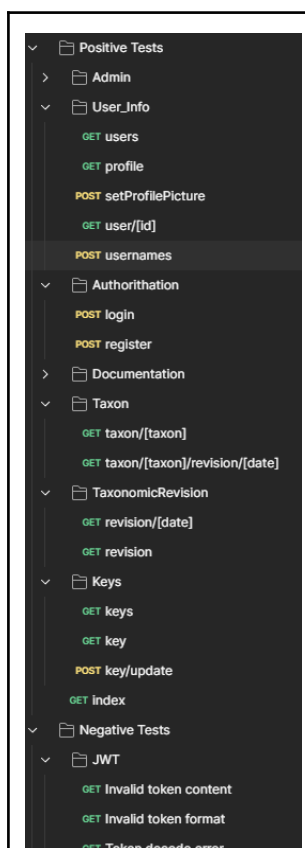
- Dentro del directorio (**var/**) se almacenan archivos que son generados y utilizados en tiempo de ejecución y dentro de (**vendor/**) están las dependencias de terceros.

3.3.4 - Bases de datos

La programación de la estructura de las bases de datos ha sido por medio de ficheros de scripts que definen la estructura de las bases de datos e insertan datos de prueba. La estructura de la base de datos de MariaDB ha sido codificada por medio de sentencias SQL. La estructura de la base de datos MongoDB ha sido generada por medio de un fichero escrito en JavaScript.

Cuando se inician los contenedores del backend, si no detectan las bases de datos, las generan por medio de los scripts anteriormente mencionados.

3.4 - Pruebas



Tests de integración de la API REST^[1] en Postman^[13].

Como es el núcleo de mi lógica de negocio, y contiene todos los casos de uso, los tests se han visto concentrados en la API REST^[1]. Dicho esto, abordar las pruebas de una API REST^[1] es un tema complejo, especialmente cuando los endpoints^[9] están diseñados para realizar lecturas o escrituras en la base de datos. Generalmente, los tests unitarios tienen como objetivo aislar la lógica del caso de uso del acceso a los datos, lo cual se logra utilizando mocks de los repositorios. Un mock es un objeto simulado que imita el comportamiento de componentes reales de manera controlada y predecible. En el caso de los repositorios, los mocks permiten definir las respuestas que estos deberían devolver, evitando el acceso real a la base de datos. Sin embargo, dado que todos mis endpoints^[9] están destinados a leer o escribir datos, el uso de mocks no resulta adecuado. Por ello, he optado por utilizar una herramienta externa, Postman^[13], para definir peticiones que interactúan de forma real con la base de datos, asegurando así que las pruebas reflejen el comportamiento auténtico de la API en un entorno real.

En mi API REST^[1], los tests han sido mayoritariamente de integración, por medio de Postman^[13], en el cual se ha elaborado una batería de tests positivos, los cuales están destinados a confirmar el correcto funcionamiento del endpoint^[9], así como una batería de tests negativos, los cuales están destinados a comprobar la gestión de los errores cuando una petición es enviada de forma incorrecta o no se dispone de permisos para esa petición en concreto. Estos tests han sido exportados a un fichero JSON que se encuentra en el directorio del Backend^[2].

4 - Posibles mejoras y ampliaciones

Mejoras sobre el desarrollo ya implementado:

- El mayor factor limitante a lo largo del proyecto ha sido el tiempo del cual se dispone, la parte del desarrollo a la cual he podido dedicar menos tiempo ha sido el frontend^[10], una de las posibles mejoras sería la refactorización de parte del código de la interfaz gráfica para redistribuir algunas funciones en nuevas clases, dejando el código más organizado.

Posibles ampliaciones del software:

- Desarrollo de un sistema de asistencia contextual que brinde definiciones de términos y diagramas inmediatos de términos y figuras taxonómicas, eliminando la dinámica de ir buscando la información dispersa en diferentes páginas.
- Marketplace de claves en la que los autores puedan poner sus claves públicas de forma gratuita o venderlas al precio indicado.
- Pestaña para solicitudes de asistencia para la identificación de morfoespecies a otros usuarios.

5 - Conclusión

El desarrollo del intérprete de claves taxonómicas ha sido una experiencia enriquecedora que ha permitido aplicar y profundizar en conocimientos académicos, cumpliendo con los objetivos iniciales y demostrando el potencial de las herramientas tecnológicas en la gestión de información taxonómica.

Personalmente, he aprendido la importancia de la planificación y gestión del tiempo bajo metodologías ágiles y a cómo encontrar fuentes de información que me ayuden a afrontar los problemas encontrados durante el desarrollo, ampliando mis conocimientos en seguridad de software, arquitectura de aplicaciones y comunicación entre sistemas, preparándome mejor para futuros desafíos profesionales.

6 - Bibliografía

Memoria:

Auto referencia: Manuel Martín Santamaría (Conocimientos o razonamiento propio, imágenes elaboradas por mi, etc...)

Kimmel, E.R., Linton, D.L., Marsico, T.D., Monfils, A.K., Morris, A.B., and Ruhfel, B.R. 2014. Taxonomic Keys. <https://collectionseducation.org/identify-specimen/taxonomic-keys>

Real Academia Española y Asociación de Academias de la Lengua Española: Diccionario panhispánico de dudas (DPD) [en línea], 2.ª edición (versión provisional).

Información para el análisis y diseño:

Metodología Scrum:

<https://scrumguides.org/scrum-guide.html>

Arquitectura hexagonal:

<https://alistair.cockburn.us/hexagonal-architecture>

Patrón Singleton:

<https://runnablepatterns.com/design/patterns/patron-singleton>

Lenguajes, tecnologías y frameworks empleados:

PHP:

<https://www.php.net/manual/en/language.operators.precedence.php>

Javascript:

<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide>

MongoDB:

<https://www.mongodb.com>

MaridDB:

<https://mariadb.org>

Docker:

<https://docs.docker.com/guides/docker-concepts/building-images/understanding-image-layers>

Framework Slim (API REST^[1]):

<https://www.slimframework.com/docs/v4>

Seguridad y tokens JWT:

<https://datatracker.ietf.org/doc/html/rfc7519>

Copilot:

<https://docs.github.com/es/copilot/about-github-copilot>

Herramientas empleadas:

Visual studio code:

<https://code.visualstudio.com>

PHP Storm:

<https://www.jetbrains.com/phpstorm>

Docker Desktop:

<https://www.docker.com/products/docker-desktop>

Postman^[13]:

<https://www.postman.com>

Draw.io:

<https://app.diagrams.net>

Paint.Net:

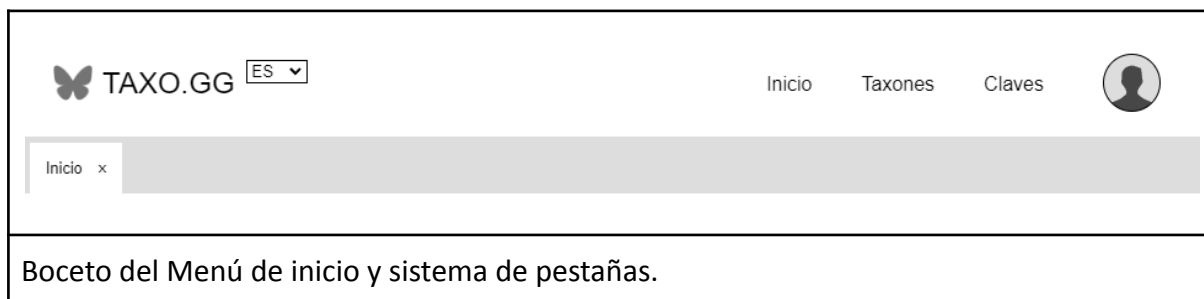
<https://getpaint.net>

7 - Anexos

7.1 - Diseño de la interfaz gráfica

Antes de programar las interfaces gráficas, he creado bocetos utilizando Paint.Net un software libre de diseño. Estos bocetos han servido como una guía visual inicial para diseñar y planificar la estructura, funcionalidad y requisitos de la interfaz.

Al hacer estos bocetos, he podido visualizar cómo se dispondrán los elementos en la pantalla y cómo interactuarán los usuarios con la aplicación, facilitando la posterior implementación.



Iniciar sesión

X

Usuario:

Usuario

Contraseña:

Contraseña

Iniciar sesión

☐ Mantener la sesión iniciada

Crear cuenta

Registrarme

X

Nombre:

Nombre

Usuario:

Usuario

Correo:

Correo

Contraseña:

Contraseña

Registrarme

¿Ya tienes cuenta?

Bocetos de los formularios de inicio de sesión y registro.

 TAXO.GG

ES

Inicio

Taxones

Claves



Inicio x

Taxones x

Perfil x

Perfil



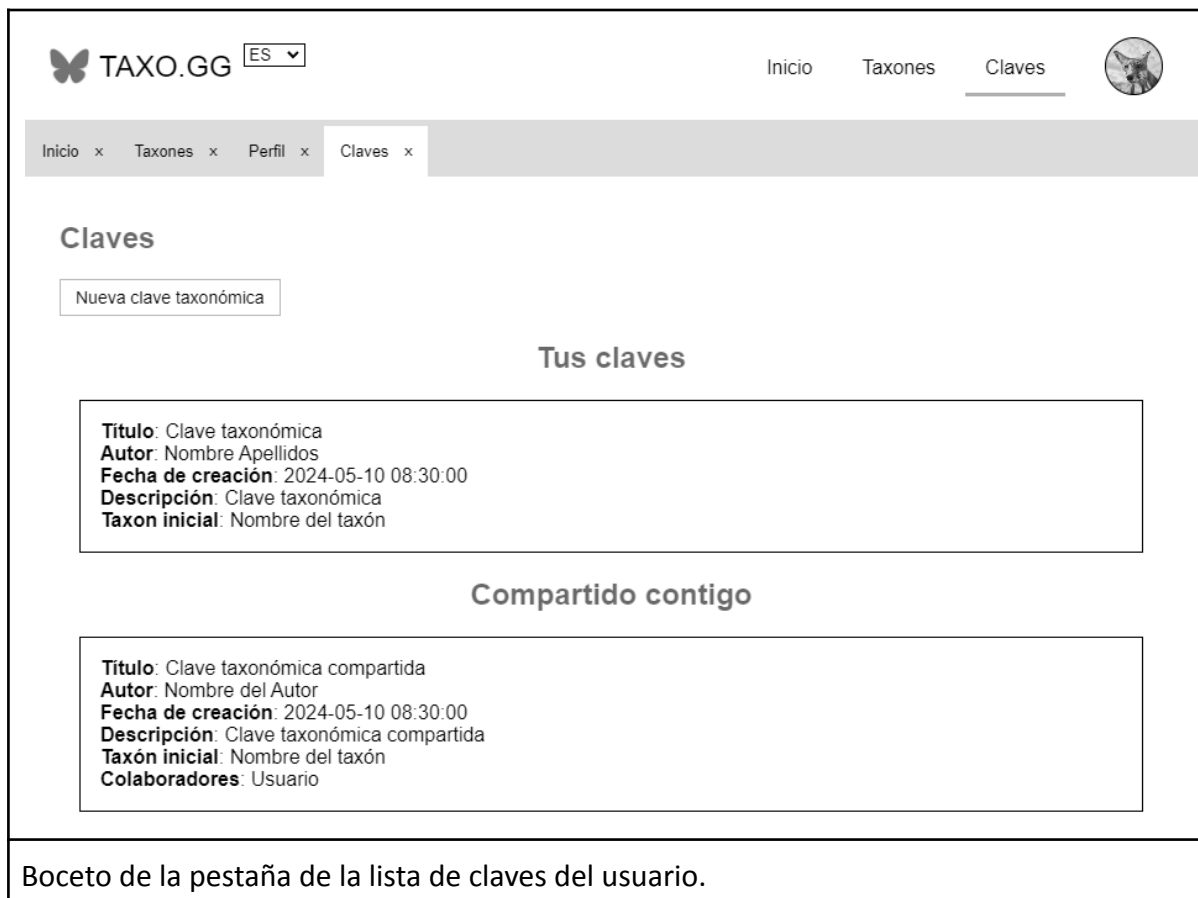
Nombre Apellidos

@username

Cerrar sesión

Cambiar foto de perfil

Boceto de la pestaña de perfil.





Id de la clave

X

Save changes

Título:

Clave taxonómica

Descripción:

Descripción de la clave taxonómica que decida el usuario

X

1

1

X

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.

4



2

X

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.

3



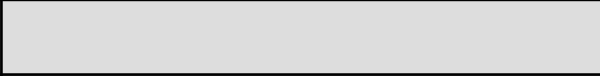
Taxón enlazado

+

Boceto de la pestaña de edición de claves.

7.2 - Paleta de colores

En el diseño de la interfaz gráfica, he elegido una paleta de colores que transmita seriedad y profesionalidad, utilizando tonos neutros y sobrios como base. Para añadir un toque de dinamismo y facilitar la visualización, he incluido el color amarillo, un color vivo que proporciona contraste y resalta elementos clave de la interfaz, como enlaces, el elemento del menú en el cual se está en ese momento, o los botones para cerrar ventanas modales o para cancelar acciones.

	#00000096
	#dddddd
	#cfb13f

7.3 - Logotipo

El logotipo de la aplicación fue diseñado por medio de Paint.Net y luego fué pasado a SVG mediante un software automático y modificado a mano para corregir imperfecciones.



7.4 - Normalización de las tablas de MariaDB

Primera Forma Normal (1NF)

La primera forma normal requiere que todos los atributos de una tabla sean atómicos.

Las tablas que he propuesto ya lo cumplen porque todas las columnas contienen valores indivisibles (no tienen atributos multivalor).

Segunda Forma Normal (2NF)

La segunda forma normal requiere que la tabla esté en 1NF y que todos los atributos no primarios dependan completamente de la clave primaria. Esto significa que cada tabla debe tener una única clave primaria y que los atributos deben depender completamente de ella.

User (id, username, name, email, pass, creation)

La clave primaria es id y todos los atributos dependen completamente de id.

Role (id, name)

La clave primaria es id y el único atributo depende completamente de id.

User_Role (↑ id_user, ↑ id_role) id_user referencia User{id} y id_role referencia Role{id}

La clave primaria es compuesta por id_user y id_role, y no tiene otros atributos.

Las tablas cumplen la segunda forma normal.

Tercera Forma Normal (3NF)

La tercera forma normal requiere que la tabla esté en 2NF y que no haya dependencia transitiva entre atributos no clave. Esto significa que los atributos no clave deben depender únicamente de la clave primaria.

Tabla User: No hay dependencias transitivas; cada atributo depende directamente de id.

Tabla Role: No hay dependencias transitivas; name depende directamente de id.

Tabla User_Role: No hay otros atributos más allá de la clave primaria compuesta.

Las tablas cumplen la tercera forma normal.

7.5 - Entornos y herramientas empleadas durante el proyecto

- Para el desarrollo de la API REST he empleado PHPStorm. Es un IDE propietario JetBrains. He podido utilizarlo de forma gratuita gracias a su modelo de licencias para estudiantes.
- Para el desarrollo del frontend^[10] he empleado Visual Studio Code. También lo he usado para la edición de los archivos de configuración de los contenedores Docker^[8]. Así como para el desarrollo de los scripts para ambas bases de datos.
- Para la elaboración de diagramas y bocetos he empleado tanto Paint.Net como draw.io.
- Para la ejecución de los contenedores he empleado Docker Desktop, ya que trabajo en windows.
- Para la ejecución del frontend^[10] he empleado XAMPP.
- Para los tests de integración he empleado postman^[13].
- Para acceder a la interfaz gráfica he usado Chrome.

7.6 - Estructura de las clases de MongoDB y optimización de consultas

Un problema muy complejo que tuve que resolver fué la optimización de las consultas de taxones^[14] a la API debido a su relación jerárquica. Para optimizar estas consultas, he implementado métodos recursivos que buscan por el campo padre (su clave foránea) de cada taxón^[14]. Estos métodos mapean toda la jerarquía de taxones^[14] desde el punto de búsqueda, realizando consultas tanto en MongoDB para el taxón^[14] buscado como para todos sus padres. Al hacer esto, la búsqueda del taxón^[14] y la de todos sus padres se almacena en caché^[5] durante 24h en la API, además también se cachean en la interfaz gráfica.

Este enfoque de doble nivel de caché^[5] tiene varios beneficios. Almacenar la jerarquía completa de taxones^[14] en la base de datos sería ineficiente y consumiría mucho espacio. En cambio, al cachear las consultas, se evita a su vez la saturación de la red y la computación excesiva de forma repetitiva e innecesaria, ya que los taxones^[14] no se actualizan con frecuencia. Además, el uso de caché^[5] tanto en la API como en la interfaz gráfica mejora el rendimiento significativamente. Si un taxón^[14] ya ha sido solicitado, la información se

recupera de la caché^[5] en lugar de volver a realizar consultas repetitivas a la API, reduciendo la carga en el servidor y acelerando el tiempo de respuesta para el usuario.

7.7 - Cómo ejecutar el proyecto

Dentro de la carpeta TFG verás dos subcarpetas, la del proyecto de backend llamada “Backend” y la del proyecto del frontend llamada “Frontend”.

El backend está configurado para ser ejecutado mediante docker, si estás trabajando en windows necesitarás Docker Desktop, no obstante requiere de tener instalado Composer en la máquina host para poder compilar la aplicación, para lo cual también hace falta tener instalado PHP, recomiendo instalarlo mediante XAMPP ya que esta aplicación también es necesaria para el frontend.

En la carpeta del backend encontrarás un ejecutable start.bat, este iniciará la compilación mediante composer y lanzará los contenedores, esto tardará unos minutos.

Para lanzar el frontend, tienes que meter la carpeta “Frontend” en la carpeta htdocs de XAMPP e iniciar el servicio de apache, tras esto podrás acceder al frontend desde un navegador web en la ruta “localhost/Frontend”.

Para detener el backend basta con hacer doble click en el archivo stop.bat de la carpeta del backend.

7.8 - Glosario

A lo largo de este documento, han sido empleados una serie de términos técnicos, a la derecha de dichos términos, de los cuales se ha considerado conveniente, aparece indicado mediante un superíndice el número de término correspondiente de este glosario.

1. **API REST:** Interfaz de comunicación entre sistemas que utiliza HTTP para la comunicación entre el frontend^[10] y backend^[2] de una aplicación.
2. **Backend:** Parte del sistema que gestiona la lógica de negocio, bases de datos y procesamiento de datos.
3. **Base de datos no relacional:** Almacena datos en formatos flexibles, como documentos y gráficos (ej. MongoDB).

4. **Base de datos relacional:** Usa tablas con filas y columnas para almacenar datos estructurados con relaciones definidas (ej. MariaDB).
5. **Caché:** Almacena temporalmente datos para reducir tiempos de acceso y mejorar el rendimiento del sistema.
6. **Contratos de comunicación:** Acuerdos formales sobre cómo enviar y recibir datos a través de una API para asegurar interoperabilidad, implementados por medio de clases o interfaces que definen la estructura del mensaje.
7. **Dicotómicas:** Divididas en dos partes, con una bifurcación, método de clasificación que consiste en dividir en dos un concepto sucesivamente.
8. **Docker:** Plataforma para desarrollar, enviar y ejecutar aplicaciones en contenedores aislados que incluyen todo lo necesario para funcionar.
9. **Endpoint:** Ruta específica dentro de una API, asociada a un caso de uso, donde se realizan operaciones como obtener o enviar datos concretos.
10. **Frontend:** Parte del sistema que interactúa con los usuarios, desarrollada con HTML, CSS y JavaScript para presentar la interfaz gráfica.
11. **Inyección de dependencias:** Patrón de diseño que permite a las clases recibir sus dependencias desde una entidad externa, promoviendo un código más modular.
12. **Token JWT (JSON Web Token):** Token estándar usado para autenticación y transferencia segura de información entre partes como un objeto JSON firmado.
13. **Postman:** Herramienta que permite desarrollar, probar y documentar APIs enviando solicitudes HTTP y verificando respuestas.
14. **Taxón:** Grupo de organismos clasificados juntos en la jerarquía taxonómica, desde el reino hasta la especie.
15. **Transpilación:** Proceso de convertir código escrito en un lenguaje de programación a otro, comúnmente usado en JavaScript moderno a versiones más antiguas.